Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

Projet : *Threads et Synchronisation — Simulation Piscine*

Réalisé par : *Ayman Bouissehak*

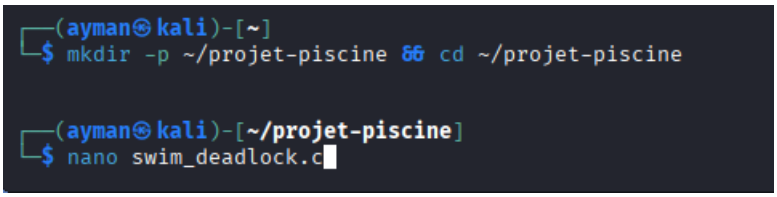
Année : 2025/2026

- **Objectif : modéliser le fonctionnement d'une piscine avec des threads et gérer la synchronisation à l'aide de sémaphores.**

- **ETAPES :**

1. **Création du dossier projet-piscine**
2. **Création des fichiers swim_deadlock.c, swim_fixed.c, Makefile, run_examples.sh**
3. **Compilation (make)**
4. **Exécution des deux versions (blocante Et corrigée)**
5. **Observation du deadlock + vérification avec ps -L -p PID**

Figure 1 — Création du répertoire de travail sous Kali Linux.



Dans cette étape, nous avons créé un dossier intitulé projet-piscine dans notre répertoire personnel à l'aide de la commande :

mkdir projet-piscine

Ce dossier servira à regrouper tous les fichiers sources du projet (codes C, Makefile, script, rapport).

Figure 2 — Code source de la version provoquant un blocage (deadlock).

Le fichier swim_deadlock.c contient une simulation où chaque baigneur (thread) tente de prendre une cabine avant un panier.
Cet ordre d'acquisition des ressources provoque un risque d'interblocage : plusieurs threads peuvent garder une cabine en attendant un panier, tandis que d'autres gardent un panier en attendant une cabine.



```c
    rand_sleep();

    logmsg("N%02d: attendre panier...", id);
    sem_wait(&sem_paniers);
    logmsg("N%02d: a un panier.", id);

    rand_sleep();
    logmsg("N%02d: libère cabine et nage.", id);
    sem_post(&sem_cabines);

    rand_sleep();
    logmsg("N%02d: revient, attend cabine pour se rhabiller...", id);
    sem_wait(&sem_cabines);
    logmsg("N%02d: a une cabine pour rhabiller.", id);

    rand_sleep();
    logmsg("N%02d: rend panier.", id);
    sem_post(&sem_paniers);

    rand_sleep();
    logmsg("N%02d: libère cabine et part.", id);
    sem_post(&sem_cabines);
    return NULL;
}

int main() {
    srand(time(NULL));
    sem_init(&sem_paniers, 0, NP);
    sem_init(&sem_cabines, 0, NC);

    pthread_t threads[NTHREADS];
    for (long i = 0; i < NTHREADS; i++)
        pthread_create(&threads[i], NULL, nageur, (void*)i);

    for (int i = 0; i < NTHREADS; i++)
        pthread_join(threads[i], NULL);

    sem_destroy(&sem_paniers);
    sem_destroy(&sem_cabines);
    return 0;
}
```

Cette version illustre le scénario incorrect, correspondant au *Scénario 1* du réseau de Petri fourni dans l'énoncé.

Figure 3 — Code source de la version corrigée (sans blocage).

Le fichier swim_fixed.c contient la version corrigée du programme.
Ici, chaque thread (baigneur) prend d'abord un panier, puis une cabine.
Une fois déshabillé, il libère la cabine pour nager, puis en reprend une pour se rhabiller.
Cet ordre d'accès aux ressources garantit l'absence de deadlock, car tous les threads suivent une même hiérarchie d'acquisition.
Cela correspond au Scénario 2 du réseau de Petri de l'énoncé.



```c
    rand_sleep();
    logmsg("N%02d: se déshabille, puis nage.", id);

    sem_post(&sem_cabines);
    rand_sleep();

    logmsg("N%02d: revient, attend cabine pour rhabiller...", id);
    sem_wait(&sem_cabines);
    logmsg("N%02d: a une cabine pour rhabiller.", id);

    rand_sleep();
    logmsg("N%02d: rend panier et libère cabine.", id);
    sem_post(&sem_paniers);
    sem_post(&sem_cabines);
    return NULL;
}

int main() {
    srand(time(NULL));
    sem_init(&sem_paniers, 0, NP);
    sem_init(&sem_cabines, 0, NC);

    pthread_t threads[NTHREADS];
    for (long i = 0; i < NTHREADS; i++)
        pthread_create(&threads[i], NULL, nageur, (void*)i);

    for (int i = 0; i < NTHREADS; i++)
        pthread_join(threads[i], NULL);

    sem_destroy(&sem_paniers);
    sem_destroy(&sem_cabines);
    return 0;
}
```

Figure 4 — Compilation réussie des programmes avec gcc -pthread.

La commande make compile automatiquement les deux fichiers swim_deadlock.c et swim_fixed.c selon les règles du Makefile.
L'option -pthread permet de lier la bibliothèque POSIX Threads utilisée pour créer et synchroniser les threads.
Cette étape valide la syntaxe du code et génère les exécutables swim_deadlock et swim_fixed

```
ayman@kali: ~/projet-piscine
Session  Actions  Edit  View  Help
  GNU nano 8.6                                  makefile *
CC = gcc
CFLAGS = -Wall -pthread -O2

all: deadlock fixed

deadlock: swim_deadlock.c
        $(CC) $(CFLAGS) -o swim_deadlock swim_deadlock.c

fixed: swim_fixed.c
        $(CC) $(CFLAGS) -o swim_fixed swim_fixed.c

clean:
        rm -f swim_deadlock swim_fixed
```

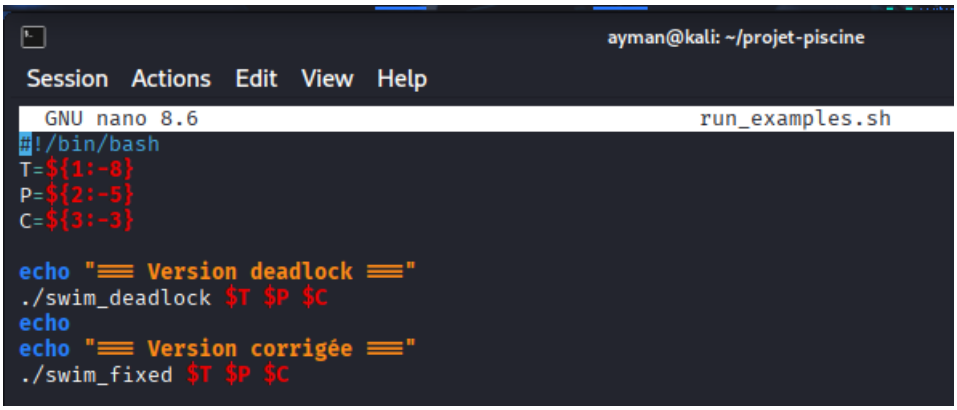
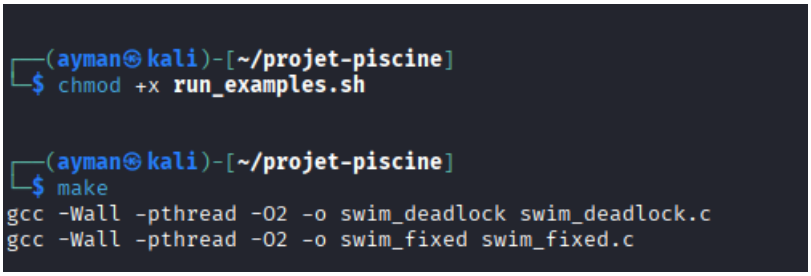


Figure 5 — Exécution de la version provoquant un interblocage.

Cette capture montre la sortie du programme ./swim_deadlock.
Au début, les threads s'exécutent normalement, mais après un certain temps, les messages cessent d'apparaître :
certains baigneurs restent bloqués sur "attendre panier..." alors qu'ils détiennent déjà une cabine.
Aucune ressource n'est libérée, et le programme ne se termine jamais.



```
[380.338] N00: arrive.
[380.338] N00: attendre cabine ...
[380.338] N00: a une cabine.
[380.339] N01: arrive.
[380.339] N01: attendre cabine ...
[380.339] N01: a une cabine.
[380.339] N03: arrive.
[380.339] N03: attendre cabine ...
[380.339] N03: a une cabine.
[380.339] N04: arrive.
[380.339] N04: attendre cabine ...
[380.339] N05: arrive.
[380.339] N05: attendre cabine ...
[380.339] N06: arrive.
[380.339] N06: attendre cabine ...
[380.339] N02: arrive.
[380.339] N02: attendre cabine ...
[380.339] N07: arrive.
[380.339] N07: attendre cabine ...
[380.525] N01: attendre panier ...
[380.525] N01: a un panier.
[380.586] N00: attendre panier ...
[380.586] N00: a un panier.
[380.591] N03: attendre panier ...
[380.591] N03: a un panier.
[380.887] N01: libère cabine et nage.
[380.888] N04: a une cabine.
[380.895] N00: libère cabine et nage.
[380.895] N05: a une cabine.
[381.020] N03: libère cabine et nage.
[381.020] N06: a une cabine.
[381.069] N01: revient, attend cabine pour se rhabiller ...
[381.193] N03: revient, attend cabine pour se rhabiller ...
[381.235] N00: revient, attend cabine pour se rhabiller ...
[381.308] N04: attendre panier ...
[381.308] N04: a un panier.
[381.315] N06: attendre panier ...
[381.316] N06: a un panier.
[381.471] N05: attendre panier ...
[381.807] N04: libère cabine et nage.
[381.807] N02: a une cabine.
[381.811] N06: libère cabine et nage.
[381.811] N07: a une cabine.
[381.912] N04: revient, attend cabine pour se rhabiller ...
[382.076] N07: attendre panier ...
[382.124] N02: attendre panier ...
[382.181] N06: revient, attend cabine pour se rhabiller ...
```

Cela illustre clairement la situation d'interblocage due à un mauvais ordre d'accès aux ressources partagées.

Figure 6 — Exécution correcte de la version sans blocage.

Dans cette exécution, les baigneurs prennent le panier avant la cabine.
Chaque thread réalise toutes ses étapes : déshabillage, baignade, rhabillage, puis libération des ressources.
Les messages "libère cabine et part." apparaissent pour tous les threads, prouvant que le programme se termine normalement.

```
[440.424] N07: arrive.
[440.424] N07: attendre panier ...
[440.524] N00: se déshabille, puis nage.
[440.524] N01: a une cabine.
[440.797] N01: se déshabille, puis nage.
[440.797] N04: a une cabine.
[440.798] N03: se déshabille, puis nage.
[440.842] N00: revient, attend cabine pour rhabiller ...
[440.842] N00: a une cabine pour rhabiller.
[440.914] N02: se déshabille, puis nage.
[441.047] N02: revient, attend cabine pour rhabiller ...
[441.047] N02: a une cabine pour rhabiller.
[441.101] N01: revient, attend cabine pour rhabiller ...
[441.116] N03: revient, attend cabine pour rhabiller ...
[441.206] N00: rend panier et libère cabine.
[441.206] N05: a un panier.
[441.206] N05: attendre cabine ...
[441.206] N05: a une cabine.
[441.346] N04: se déshabille, puis nage.
[441.346] N03: a une cabine pour rhabiller.
[441.530] N02: rend panier et libère cabine.
[441.530] N06: a un panier.
[441.530] N06: attendre cabine ...
[441.530] N01: a une cabine pour rhabiller.
[441.580] N04: revient, attend cabine pour rhabiller ...
[441.705] N03: rend panier et libère cabine.
[441.706] N06: a une cabine.
[441.706] N07: a un panier.
[441.706] N07: attendre cabine ...
[441.758] N05: se déshabille, puis nage.
[441.758] N04: a une cabine pour rhabiller.
[441.792] N01: rend panier et libère cabine.
[441.792] N07: a une cabine.
[441.889] N05: revient, attend cabine pour rhabiller ...
[442.075] N04: rend panier et libère cabine.
[442.075] N05: a une cabine pour rhabiller.
[442.121] N06: se déshabille, puis nage.
[442.155] N07: se déshabille, puis nage.
[442.289] N06: revient, attend cabine pour rhabiller ...
[442.290] N06: a une cabine pour rhabiller.
[442.531] N07: revient, attend cabine pour rhabiller ...
[442.531] N07: a une cabine pour rhabiller.
[442.617] N05: rend panier et libère cabine.
[442.685] N06: rend panier et libère cabine.
[443.091] N07: rend panier et libère cabine.
```

Conclusion

Ce projet a permis de comprendre les principes de la programmation concurrente et la gestion de la synchronisation entre threads.
À travers la simulation d'une piscine, nous avons observé une situation d'interblocage dans la version initiale et avons proposé une solution efficace à l'aide des sémaphores POSIX.
Cette expérience illustre l'importance de l'ordre d'acquisition des ressources dans les systèmes parallèles.